A high-performance Delta lake for Drishti on AWS

Past 200 TB and at bank scale: the lake in S3, the risk lake, JSON Lines in, the scripts, the keys, the partitions and every optimisation

Drishti 2.9.4 · everything shipped up to 2.9.4 is built · only scaling VaR out across a cluster of Drishti servers is designed, not built

AWS scripts tested with --help and --dry-run; Delta on S3 and lake publish tested on S3 simulators; first run in a real AWS account still to do

Drishti · Any data. Any domain. One grammar.

Second edition, 2026-10-27 (first edition 2026-10-20, Drishti 2.6.5)

Copyright © 2026 Ashutosh Sinha. All rights reserved. Proprietary and confidential.

What this deck covers

contents

1 The picture, and Delta or Iceberg

2 Why S3 and not EFS for the lake

3 Built, designed, proposed

4 First-time setup, steps 0–11

5 The layout: buckets, prefixes, partitions

6 Keys: id, promoted, vectors

7 Files: sizes, row groups, sorting

8 Data in: JSON Lines, then Delta

9 Loading: daily, backfill, restatement

10 S3: safe commits and three simulators

11 Maintenance: retention, 7-year floor, tiers

12 The risk lake: VaR and exposure

13 Bank scale: 2.3M positions, 15k portfolios

14 Caches: EFS or local NVMe

15 Staying fast: history, Drishti*net, alarms

16 The demo lake, public data, DATAMAP

17 Sizing and cost

18 Runbooks

19 The scripts

20 Limits, decisions, related documents

The one rule: the lake lives in Amazon S3

Every table's Parquet data files AND its _delta_log are in S3. Nothing of the lake is on EFS.

EFS holds only what a node must find again after a restart: configuration, users and tokens, Drishti*net state, the content store's filesystem backend if chosen, and the caches that should be warm after a restart.

- **S3 (the lake):** `s3://<org>-drishti-<env>-lake/lake/<domain>/<table>/` with `business_date=…/PRODUCT_TYPE=…/part-….snappy.parquet` and `_delta_log/`
- **Domains in 2.9.4:** trading, reference, legal, collateral, credit, market, macro, news, civic, **risk** (VaR and exposure inputs), **risk-results** (official results, append-only), us-equities, gleif, funds
- **S3 (landing):** the sources' dumps as JSON Lines, `<date>/<source>/*.jsonl.gz`, kept 30 days (`LANDING_KEEP_DAYS`)
- **S3 (logs):** access logs, SSE-S3, expire after 400 days
- **EFS (state):** `/drishti/<env>/<node>/opt/drishti/{data,config,sutras}`, one access point per node per path, identical path inside the container
- **EFS (loader scratch):** one day of one table for the minutes of a load, then removed; the per-table lock files
- **Never on EFS:** lake data, `_delta_log` (EFS is a staging place only, for tables the server's own writer builds)

What changed for Delta and AWS since 2.6.0

new in this edition

Release	What changed for the lake and AWS	Slides
2.6.5	History queries over the lake: `TRD <id> 30`; the Delta fast path lists an entity's days from partitions and id maps	78
2.6.7	Delta and Iceberg side by side (`--store calypso=iceberg`); Drishti*net gossip on its own port and HTTP/2 between nodes; `lake maintain` lays out the per-source trade tables; `lake build` one commit per business date	9, 27, 58
2.7.0	The content store (filesystem, sqlite, postgres, **s3** with conditional PUT); S3 tested on S3Mock, moto and MinIO; `lake publish` checks the store it commits to; us-equities and gleif served from Delta	53, 54, 85
2.8.0	VaR computed for any scope from P&L vectors in Delta (`list<double>` columns); risk lake kept 7 years; official VaR published append-only to `risk-results/var-official`	62, 63, 65
2.9.0	Exposure computed for any scope from `exposure-path` vectors; official exposure published; VaR and exposure in BI read the published results	67
2.9.2–2.9.3	The funds pack (SEC public data, on Delta); DATAMAP: every kind, count and join checked against the lake	85, 86
2.9.4	JSON Lines is the data standard (generate, then load); CSV input deprecated; official results also as JSONL beside Delta; scenario PVs, intraday batches and Kafka micro-batches appended to the lake; the demo lake has 84 tables	44, 64, 84

Status: what exists today, and how it was tested

BUILT	In the code and tested; shipped in a release up to 2.9.4. Everything on the AWS lake, the risk lake, JSON Lines, the S3 work and Drishti*net in this deck is built.
DESIGNED	A design its own document describes and says is not built: SUPER_SCALE steps S1–S8; VaR sharding across a cluster of Drishti servers; the bank-scale vector layout (sort, ZSTD, byte-stream split).
PROPOSED	This deck's recommendation, not yet in the docs (each one is marked where it appears).
MEASURED	A number measured on Drishti (the doc names the test or benchmark).
ESTIMATED	A number from arithmetic or a linear extrapolation of measured points; to be replaced by measurement.

How it was tested

Every AWS script with --help and --dry-run (`tools/test_deploy_aws.py`, an aws that fails if called). Delta on S3 and `lake publish` on moto, including three publishes at once; the content store and the Iceberg engine on S3Mock and MinIO; mixed Delta and Iceberg storage by an equality test (18 answers the same).

Not yet: the first run in a real AWS account (steps 0–11 and runbooks 18). Every number marked ESTIMATED waits for it.

01
The picture

From the source systems' JSON Lines dumps to a trader's screen: landing bucket → loader task → the lake in S3 → Drishti servers → EFS for state; and Delta or Iceberg underneath, with the same Drishti above

The picture: the whole flow on one page

Sources

- ENDUR, CALYPSO, MUREX, reference, risk batch
- JSON Lines `.jsonl(.gz)`; CSV deprecated

S3 · <org>-drishti-<env>-landing

- `2026-10-20/endur/*.jsonl.gz`, …/calypso/, …/murex/
- kept `LANDING_KEEP_DAYS` (30), then expired

EventBridge Scheduler

- 22:30 New York, Monday to Friday
- runs the loader task; nightly maintain, daily report

ECS task · loader · load-day.sh, per table, under the table's lock (EFS)

- 1 `aws s3 sync` landing → EFS scratch 2 `lake build` → scratch Delta table: promoted and vector columns, sorted by id, row groups 1 MB
- 3 `aws s3 sync` data files → S3 (new UUID names) 4 `lake publish` ONE commit (conditional PUT) 5 metrics Drishti/Lake + data-load signal

S3 · <org>-drishti-<env>-lake (THE LAKE)

- `lake/<domain>/<table>/_delta_log/` + `business_date=…/` Parquet
- domains: trading … market, **risk**, **risk-results**, public data
- versioning · SSE-KMS · VPC endpoint only · tiers by business date

ECS services drishti-a, -b, -c (server + console)

- Delta connectors, native engine, ranged GETs from `DRISHTI_DELTA_ROOT`
- VaR and exposure engines read `risk/`; caches in heap and on disk
- Drishti*net: gossip port + HTTP/2 between task IPs

ECS task · maintenance (nightly)

- checkpoint, retention (7-year floor on risk), vacuum; lifecycle rules

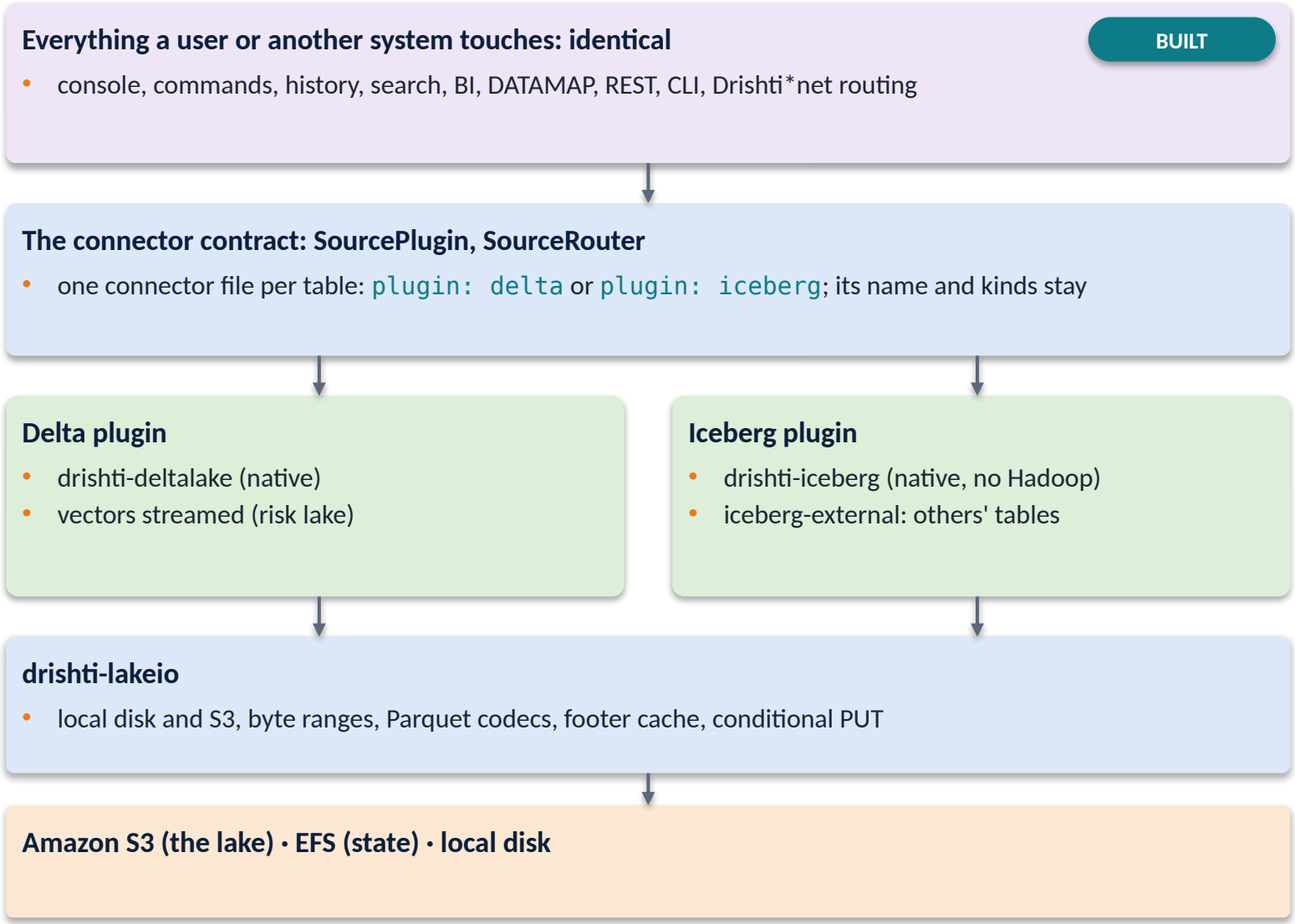
EFS (Regional, encrypted, Elastic)

- state, caches, loader scratch, locks; NO lake data

The picture, in words: each box

Box	What it is	Key facts
Sources	ENDUR, CALYPSO, MUREX, reference systems, the risk batch	send JSON Lines, gzipped or not; one folder per date and source; CSV input deprecated in 2.9.4
Landing bucket	`<org>-drishti-<env>-landing`	`<date>/<source>/…`; dumps expire after 30 days (`LANDING_KEEP_DAYS`); same protections as the lake
Scheduler	EventBridge Scheduler	`LOAD_SCHEDULE` 22:30 in `LOAD_SCHEDULE_ZONE` (New York), Mon–Fri; also -maintain and -report
Loader task	ECS task `drishti-<env>-loader`	load-day.sh: sync → build → sync → publish → metrics, per table, under the table's EFS lock; scratch always removed
The lake	`<org>-drishti-<env>-lake`	Delta tables: Parquet + `_delta_log`; versioning, SSE-KMS, VPC endpoint only, tiers by business date
Server nodes	ECS services `drishti-a/-b/-c`	server + console per task; ranged GETs; VaR and exposure engines; Drishti*net gossip port + HTTP/2
Maintenance task	ECS task, nightly (`MAINTAIN_SCHEDULE`)	`lake maintain` per table under its lock (risk domains never below 7 years), then setup-lifecycle.sh
EFS	Regional, encrypted, Elastic throughput	`/drishti/<env>/<node>/opt/drishti/{data,config,sutras}`; scratch and locks; never lake data

Delta or Iceberg: the architecture above storage is identical



The same on both

- pages, searches, history, BI: 18 of 18 answers equal in the mixed-storage test
- commits: S3 conditional PUT, probed at start
- S3, IAM, KMS, ECS, EFS and Cloud Map set-up

Where they differ today

- `deploy/aws/delta-lake/`: load, publish, maintain, lifecycle are Delta's
- VaR and exposure inputs: Delta only (streamed vectors); on Iceberg not built
- official results: Delta on a local or EFS root (+ JSONL); to S3 PROPOSED

The companion deck: Drishti on AWS with Apache Iceberg

COMPANION DECK

Drishti on AWS with Apache Iceberg

Drishti-AWS-Iceberg.pptx · 77 slides

Read it with this deck: the same picture, the same S3-not-EFS answer, the same Drishti above the storage; Iceberg's own catalog, files, maintenance and a guide to choosing.

What it covers

- drishti-iceberg as built: native engine, no Hadoop; path-based tables and REST catalogs (SigV4 for Glue)
- deploying on AWS with Iceberg: catalog, IAM, KMS, network
- layout, files, loading and maintenance in Iceberg terms
- **Delta Lake and Iceberg side by side**, and **which to choose: a guide for a bank**
- risk workloads: VaR and exposure inputs and official results are Delta BUILT (local/EFS), Iceberg PROPOSED
- migration and coexistence
- `iceberg-external`: reading tables Spark, Flink, Trino or Glue write

This deck stays on Delta: the AWS scripts, the risk lake and the bank-scale sizing are Delta's. Choose the format per connector; slide 9 shows why nothing above it changes.

02
Why S3 and not EFS for the lake

Cost, request rate, latency, durability, tiers, other readers and writers, compared at 200 TB

Why S3 and not EFS: the comparison

	Amazon S3	Amazon EFS
Cost of 200 TB (list prices, one Region)	about **USD 1,400 a month** tiered by business date; USD 4,600 all in Standard	**USD 60,000 a month** in EFS Standard; USD 3,200 in Infrequent Access plus a charge on every byte read
Request rate	at least 3,500 writes and 5,500 reads a second per prefix; every `business_date=…/PRODUCT_TYPE=…/` folder is a prefix	one file system's limits, shared by every node, loader and maintenance job
Latency of a single read	10–30 ms to the first byte in-Region; Drishti reads one row group	about 1 ms for hot data in Standard; tens of ms in IA
Durability	eleven nines, ≥ 3 AZs, versioning, Cross-Region Replication	Regional (three zones); backup by AWS Backup
Tiers	Standard → Standard-IA → Glacier Instant Retrieval (still ms), by date	Standard → IA → Archive by last access only
Other readers	Athena, Trino, Spark, EMR, Glue read the same Delta tables	only something that mounts the file system
Writers	conditional writes make concurrent commits safe	a hard link on a POSIX file system

At 200 TB+ EFS costs 10–40× more and shares one throughput budget. Drishti reads one day's files, one row group of one file, by byte ranges: built for object storage. **The lake is in S3; EFS keeps the small, hot, per-node state.**

03
Built, designed, proposed

What runs today (everything up to 2.9.4), what its own documents describe as designed, and what this deck proposes

Built (1/2): the lake on S3, as in the first edition

Capability	Status	Where
Reading Delta tables in S3: file lists from the last checkpoint, id maps, one row group per read, footers cached, documents cached on disk	BUILT	drishti-deltalake, drishti-lakeio, plugins/drishti-plugin-delta
`lake build`: JSON Lines into a Delta table on local disk or EFS: promoted and vector columns, partitions, sorted by id per batch	BUILT	tools/lake/build.py, the Java writer in the server jar
`lake publish`: a built table committed into an S3 table as ONE commit (S3 conditional writes)	BUILT	tools/lake/publish.py, test_publish.py
`lake maintain` on `s3://`: retention, checkpoint, vacuum; relayout of the per-source trade tables (fixed in 2.6.7)	BUILT	tools/lake/maintain.py (`--force-foreign`)
Buckets, IAM, conditional-write check, lifecycle tiers, alarms, report, load, backfill, maintenance, EFS, Cloud Map, task definitions, schedules	BUILT · dry-run tested	deploy/aws/
ECS images (server, console, loader) and entry points	BUILT · files	deploy/docker/ecs/
Drishti*net shards by table; replicas send one entity to the same node by a hash of its id	BUILT	DRISHTI_NET.md §7, §10

Built (2/2): added since 2.6.0

Capability	Release	Where
History queries over the lake (`TRD <id> 30`, `knownAt`), the Delta fast path, across Drishti*net	2.6.5	HISTORY_QUERIES.md, DELTA_CONNECTOR.md §8
Delta and Iceberg side by side, per source system; one connector per system	2.6.7	build_demo_lakes.py `--store`
Drishti*net gossip on its own port; peer data over HTTP/2	2.6.7	NET.md §4, §7
Content store with an S3 backend (conditional PUT, ETag); S3 on S3Mock, moto, MinIO	2.7.0	CONTENT_STORE.md, TOOLS.md §4.12
Vector columns `list<double>`; VaR inputs in `risk/`; 7-year floor; official VaR append-only	2.8.0	VAR_GUIDE.md §15, §19, §29
`exposure-path` vectors; official exposure; VaR and exposure in BI from the published results	2.9.0	EXPOSURE_GUIDE.md §15, §20
DATAMAP: kinds, counts and joins checked against the lake	2.9.3	USER_GUIDE.md, `/api/v1/datamap`
JSON Lines standard; CSV deprecated; official results as JSONL; scenario PVs; intraday and Kafka batches	2.9.4	DATA_FORMATS.md, VAR_GUIDE.md §8.4, §15.9

Designed, not built: what their own documents say

Item	What it adds	Why it matters at scale
S1	Range maps from the log's per-file id statistics; Parquet Bloom filters on id	find an id without building a day's id map
S2	Tables by year (`root: …/{year}`), closing a year	a table's checkpoint stops growing with seven years of live files
S3	A loader fleet writing slices of one table at once; a day register	loads of very large days finish in the window
S4	Products (link indexes, group totals, rollups, id lists) written with the data	aggregates and links without scanning
S5, S6	Each server owns a share of (table, day) slices on the ring	N servers hold N × one server's working set
S7, S8	Warm tier of memory-mapped column files; benchmark at scale in AWS	hot days from memory; measured numbers
VaR sharding	Each server maps only its ranges of a VaR date; partial vectors added in a fixed order	one VaR date larger than one server
Vector layout	Rows sorted by legal entity, desk, book; ZSTD + byte-stream split; ~512 MB files	a node is a contiguous run of rows

DESIGNED

Proposed by this deck: not yet in the docs

#	Proposal	Why
1	Set node.yaml `gossip-port` to a fixed number on ECS and open it in the `drishti-node` security group	`auto` changes the port at each start; behind a security group gossip falls back to 18480
2	Official results (`risk-results/`) written to an EFS root, then copied to S3 by `lake publish` after each capture	the publishers' Java writer does not write `s3://` (publishError)
3	Re-enable `compact` for the S3 trade tables after a relayout test on S3	the 2.6.7 fix lays out per-source tables
4	`exposure-path` partitioned by date and run, sorted by netting set (as EXPOSURE_GUIDE.md proposes)	a netting set in one streaming pass
5	A published Delta table of node and portfolio vectors per currency (from the VaR deck)	BI and peers read sums, not 2.3M rows
6	Change data feed on the risk input tables (from the VaR deck)	followers see intraday deltas

PROPOSED

04

First-time setup, step by step

Twelve steps (0–11). Each: the script, the exact command, what you should see. Always --dry-run first, read it, then run

Setup at a glance: steps 0 to 11

0 Prerequisites

VPC, endpoints, EFS, KMS, ECS, ALB, SNS, secrets

1 Settings

`deploy/aws/env from env.example`

2 Buckets

`setup-buckets.sh`

3 IAM

`setup-iam.sh`

4 Prove commits safe

`check-conditional-writes.sh`

5 Storage tiers

`setup-lifecycle.sh`

6 EFS access points

`ecs/setup-efs.sh`

7 Cloud Map seeds

`ecs/setup-cloud-map.sh (twice)`

8 Images

docker build ×3, push to ECR

9 Tasks, services, schedules

`ecs/register.sh`

10 First load

`load-day.sh --date …`

11 Alarms and report

`setup-alarms.sh, report.sh`

Step 0: prerequisites (the platform team's templates)

- A VPC with private subnets in three Availability Zones
- A gateway VPC endpoint for S3 on their route tables (lake traffic never leaves the VPC; the bucket policy requires it)
- Interface endpoints for ECR, CloudWatch Logs, Secrets Manager and STS
- An EFS file system: Regional, encrypted, Elastic throughput, a mount target per subnet
- A KMS key for the lake; its key policy allows the roles of step 3
- An ECS cluster
- An Application Load Balancer with a target group for the consoles on port 17480
- An SNS topic for alarms
- The four secrets of AWS_DEPLOYMENT.md §11.3 (token secret, session secret, cluster key, admin seed)

Who builds these?

The platform team, with its own CloudFormation or Terraform templates. The Drishti scripts take their ids (subnets, file system, key ARN, cluster, target group, topic) from `deploy/aws/env`.

Run the scripts from

a workstation or admin host with AWS CLI v2, the repository, Java 21 and `drishti-console/.venv`.

Content store on S3 (optional, 2.7.0): its bucket needs conditional writes too (step 4).

Step 1: the settings file


```
cp deploy/aws/env.example deploy/aws/env        # git-ignored; holds account ids and names, never a secret
$EDITOR deploy/aws/env                          # replace every <placeholder>
cp deploy/aws/delta-lake/tables.example deploy/aws/delta-lake/tables.conf   # your tables; set LAKE_TABLES_FILE
```

You should see: nothing yet. A script run with a placeholder left names it: `setup-buckets.sh: LAKE_KMS_KEY_ARN is still a placeholder (<arn:aws:kms:…>) in deploy/aws/env`

```
trading/trade_endur     trade   config/packs/trading   trade-endur     {date}/endur/          -      PRODUCT_TYPE
```

Field	Example	Meaning
folder	`trading/trade_endur`	the table's folder under the lake prefix (domain / table)
kind	`trade`	the Drishti kind
pack	`config/packs/trading`	the pack whose layout (key, promoted and vector columns) is used
connector	`trade-endur`	the pack's connector that reads this table
landing prefix	`{date}/endur/`	where the JSON Lines dumps are; `{date}` is the business date
business-date field	`-`	the field holding the date, or - for the day being loaded
partition column	`PRODUCT_TYPE`	the second partition key

Step 2: the buckets


```
deploy/aws/delta-lake/setup-buckets.sh --dry-run
deploy/aws/delta-lake/setup-buckets.sh
+ aws s3api create-bucket --bucket acme-drishti-prod-lake --object-ownership BucketOwnerEnforced \
      --create-bucket-configuration LocationConstraint=eu-west-2
+ aws s3api put-bucket-versioning --bucket acme-drishti-prod-lake --versioning-configuration Status=Enabled
+ aws s3api put-bucket-encryption --bucket acme-drishti-prod-lake --server-side-encryption-configuration file://…
+ aws s3api put-bucket-policy --bucket acme-drishti-prod-lake --policy file://…/policy-….json
setup-buckets.sh: done: acme-drishti-prod-lake, acme-drishti-prod-landing, acme-drishti-prod-logs
```

- **Three buckets:** `-lake` (every Delta table, the risk lake included), `-landing` (JSON Lines dumps, 30 days), `-logs` (access logs, 400 days)
- **On the lake:** versioning on; SSE-KMS with S3 Bucket Keys; Block Public Access; bucket-owner-enforced ownership; TLS only; VPC endpoint only (plus `drishti-<env>-admin*` roles); access logs; request metrics on `lake/`
- **Safety net:** noncurrent versions expire after 14 days; incomplete multipart uploads abort after 2 days
- **Idempotent:** run again, it says `acme-drishti-prod-lake exists: kept, settings applied again`
- `--dry-run` prints each changing call as `+ aws …` and each read as `? aws …`; none is made

Step 3: who may do what (IAM)


```
deploy/aws/delta-lake/setup-iam.sh --dry-run
deploy/aws/delta-lake/setup-iam.sh
```

Role	Policy (deploy/aws/delta-lake/iam/)	May
`drishti-<env>-server-task`	`server-policy.json`	read the lake prefix (GET, LIST), decrypt; mount EFS through access points
`drishti-<env>-loader-task`	`loader-policy.json`	read the landing bucket; write data files and commits; never delete (an explicit deny)
`drishti-<env>-maintenance-task`	`maintenance-policy.json`	checkpoints, retention and vacuum: the only role that deletes; the lifecycle rules
`drishti-<env>-execution`	AmazonECSTaskExecutionRolePolicy + `drishti/<env>/*` secrets	ECS pulls images, writes logs, injects secrets
`drishti-<env>-scheduler`	inline	EventBridge Scheduler runs the lake tasks

You should see `setup-iam.sh: done: three policies, five roles (…)`. The servers only read the lake: what a server writes itself (official results, Kafka intraday batches) needs its own path to S3 (slide 65).

Step 4: prove that commits are safe


```
deploy/aws/delta-lake/check-conditional-writes.sh
check-conditional-writes.sh: ok: the second create was refused (412 PreconditionFailed): concurrent commits are safe
```

- **What it does:** creates a test object with `If-None-Match: *`, then tries again; S3 must refuse the second with 412
- **Why it matters:** a Delta commit is the creation of `_delta_log/<version>.json`; two writers must never both create one version
- **Any other answer: stop.** A store or proxy that drops the header cannot guarantee safe concurrent commits; then only one writer per table (the scripts' per-table locks give that if nothing else writes)
- **No DynamoDB lock table:** deltalake 1.x commits with conditional PUT

The same guarantee, used three times

- **Delta commits:** `lake publish`, `lake maintain` (deltalake 1.x)
- **Iceberg commits:** drishti-lakeio's conditional PUT, probed at start (a store without it: single writer)
- **Content store on S3:** create `If-None-Match: *`, update `If-Match: <ETag>`; a lost race is `409 DRS-5057`

Step 5: storage tiers by business date


```
deploy/aws/delta-lake/setup-lifecycle.sh --print | head -20     # the rules, no AWS call
deploy/aws/delta-lake/setup-lifecycle.sh
```

You should see `setup-lifecycle.sh: 129 rules for acme-drishti-prod-lake` (three tables from 2019).

- **By date, not by age:** age is the time since an object was written, wrong for a restated or backfilled day; one rule per table per business month, with transition dates
- Standard → Standard-IA when the month's last day is 90 days old (`TIER_IA_AFTER_DAYS`); → Glacier Instant Retrieval at two years (`TIER_GLACIER_IR_AFTER_DAYS`)
- Never Glacier Flexible Retrieval or Deep Archive (hours to restore); never an S3 expiry of current lake objects
- `_delta_log/` never matches: commits and checkpoints stay in Standard; objects under 128 KB stay where they are
- Rules exist three months ahead; maintain.sh rewrites them nightly; S3 allows 1,000 rules and the script refuses more
- **Risk tables:** 7 years kept, old months in Glacier IR, still read in milliseconds (official numbers stay reproducible)

Step 6: EFS access points, one per node per path


```
deploy/aws/ecs/setup-efs.sh --dry-run
deploy/aws/ecs/setup-efs.sh
```

You should see one access point id per line (`fsap-0a1b…`) and `setup-efs.sh: 14 access points on fs-… for three nodes`.

Owner	Access points	Root (identical path inside the container)
each node (drishti-a, -b, -c)	data, config, sutras	`/drishti/<env>/<node>/opt/drishti/<path>` → `/opt/drishti/<path>`
consoles	data, config, sutras	`/drishti/<env>/consoles/opt/drishti/<path>`
loader	data	`/drishti/<env>/loader/opt/drishti/data` (scratch lakes, table locks)
maintenance	data	`/drishti/<env>/maintenance/opt/drishti/data`

- Every access point runs as uid/gid 10001 (the image's user); 3 × 3 + 3 + 1 + 1 = 14
- Idempotent: the access point's name is its client token, so running again returns the same ids
- The rule from AWS_DEPLOYMENT.md: every path a container writes maps to the identical path on EFS

Step 7: Cloud Map seeds and the Drishti*net transport


```
deploy/aws/ecs/setup-cloud-map.sh        # once for the namespace (it takes a minute) …
deploy/aws/ecs/setup-cloud-map.sh        # … and again for one service per node
```

- ECS registers each node's task under `drishti-a.drishti.local` with the task's own address
- Servers advertise `http://<task IP>:18480`, never loopback (refused since 2.6.0)
- A new node finds the cluster through its seed's Cloud Map name, then learns the rest by gossip
- The loader's data-load signal (`DRISHTI_SIGNAL_URL`) also uses a node's Cloud Map name

Gossip on its own port (2.6.7) — BUILT

- node.yaml `gossip-port`: auto · a number · off
- busy nodes never suspected: 12 s saturated, no suspicion

HTTP/2 between nodes (2.6.7) — BUILT

- routing.yaml `peer-protocol: http2` (h2c), one connection per peer
- 1 connection instead of 227; p99 117 ms vs 370 ms

On ECS: a fixed gossip port — PROPOSED

- set `gossip-port` to a number (e.g. 18490) on every node
- open it node to node in the `drishti-node` security group

Step 8: the images


```
./mvnw -q clean package -DskipTests
docker build -f deploy/docker/ecs/server.Dockerfile  -t "$ECR_REGISTRY/drishti-server-ecs:2.9.4" .
docker build -f deploy/docker/ecs/console.Dockerfile -t "$ECR_REGISTRY/drishti-console-ecs:2.9.4" .
docker build -f deploy/docker/ecs/loader.Dockerfile  -t "$ECR_REGISTRY/drishti-loader-ecs:2.9.4" .   # carries env
docker push …                                         # each of the three, after aws ecr get-login-password
```

Image	Built from	Entry point checks / role
`drishti-server-ecs`	`deploy/docker/ecs/server.Dockerfile`	server-entrypoint.sh: EFS mounted and writable, real IPv4 (never loopback), first-start node.yaml and peers; health check
`drishti-console-ecs`	`deploy/docker/ecs/console.Dockerfile`	console-entrypoint.sh: the console, a separate container from the server
`drishti-loader-ecs`	`deploy/docker/ecs/loader.Dockerfile`	loader-entrypoint.sh: EFS mounted and writable; runs the lake scripts; copies deploy/aws/ with your env (no secret)

- Loader tasks read the same settings: `DRISHTI_AWS_ENV=/opt/drishti/app/deploy/aws/env`; rebuild the loader image when they change
- The loader image carries the Python lake tools (`lake build`, `lake publish`, `lake maintain`, `drishti.py convert`)
- The build context is the repository root; nothing is built or run on a developer laptop by the scripts

Step 9: task definitions, services and schedules


```
deploy/aws/ecs/register.sh --dry-run
deploy/aws/ecs/register.sh                          # --nodes --lake-jobs
deploy/aws/ecs/register.sh --consoles 4             # optional: a console service of its own
```

You should see: a task definition and a service per node (`drishti-prod-drishti-a`, …), the loader, maintenance and report task definitions, three schedules (`drishti-prod-load-day`, `-maintain`, `-report`), then `register.sh: done`.

- The node task sets `DRISHTI_DELTA_ROOT=s3://<lake bucket>/lake`: every shipped Delta connector reads the lake in S3 with no edit, **the risk lake's** `risk-store`, `exposure-store`, `var-results` and `exposure-results` included
- The official-results publishers default to the same root and do not write `s3://`: give them an EFS root (slide 65)
- Templates: `deploy/aws/ecs/templates/` → node-task, node-service, console-task, console-service, lake-task, schedule-target
- `--cache nvme` (EC2 launch type) mounts the host's `/mnt/nvme/drishti/<node>` at `/opt/drishti/nvme` (section 14)
- Schedules: `LOAD_SCHEDULE` (22:30 Mon–Fri), `MAINTAIN_SCHEDULE` (Tue–Sat 02:30), `REPORT_SCHEDULE` (daily)

Step 10: the first load


```
deploy/aws/delta-lake/load-day.sh --dry-run --date 2026-10-20 --tables trading/trade_endur
deploy/aws/delta-lake/load-day.sh --date 2026-10-20 --tables trading/trade_endur
```

You should see the five steps of section 9, then:

```
lake publish: ok exit=0 published <n> file(s), <bytes> bytes, dates 2026-10-20 to s3://…/trade_endur version=0 (new table)
load-day.sh: business date 2026-10-20: exit 0
```

On ECS the same runs as the loader task:

```
aws ecs run-task --task-definition drishti-prod-loader \
  --overrides '{"containerOverrides":[{"name":"loader","command":["deploy/aws/delta-lake/load-day.sh","--date","2026-10-20"]}]}' …
```

In the console

`TRD <id>` on 2026-10-20 opens the trade; **Admin → Health** shows `trade-endur UP (engine: native)`; `DATAMAP trade` counts the day's trades and checks their joins against the lake.

The dumps are JSON Lines (2.9.4)

One object a line, `.jsonl.gz`. JSON arrays still load; a CSV export is converted first: `drishti.py convert trades.csv` (CSV input is deprecated).

Step 11: alarms and the report


```
deploy/aws/delta-lake/setup-alarms.sh
deploy/aws/delta-lake/report.sh
```

You should see `setup-alarms.sh: 17 alarms put` (four per table, two for the bucket and EFS, one per node) and the report of section 15.

Per table (× 3 = 12)

- load-failed
- not-loaded
- load-slow
- log-long

Bucket and EFS (2)

- lake-s3-5xx
- efs-io-limit

Per node (× 3)

- <node>-down

- All notify `ALARM_SNS_TOPIC_ARN`; details on slide 82
- `report.sh` is also scheduled daily (`REPORT_SCHEDULE`) and sends its numbers to CloudWatch
- Drishti*net health (gossip port, peer protocol) is on **Admin → Health** and **Admin → Net**

05
The layout

Buckets, prefixes, tables and partitions: what goes where in S3, and why; the risk lake's domains beside the documents

5.1 Buckets

Bucket	Holds	Settings (setup-buckets.sh)
`<org>-drishti-<env>-lake`	every Delta table: data files and `_delta_log`; the risk lake; JSONL copies of official results	versioning on; SSE-KMS with S3 Bucket Keys; Block Public Access; owner-enforced; TLS only; VPC endpoint only; access logs; request metrics on `lake/`; noncurrent versions 14 days; multipart aborts after 2 days
`<org>-drishti-<env>-landing`	the sources' JSON Lines dumps	the same protections; dumps expire after 30 days
`<org>-drishti-<env>-logs`	S3 access logs	SSE-S3; expire after 400 days

Versioning on, with a short noncurrent window.

Delta keeps its own history (time travel back to the vacuum window; 7 years for the risk domains), so versions are a safety net, not history. A file deleted by mistake (a wrong vacuum, a script against the wrong prefix) stays recoverable for `NONCURRENT_KEEP_DAYS` (14); then the lifecycle expires it.

5.2 Prefixes and tables

Choice	Why
One table per source system and kind (`trade_endur`, `_calypso`, `_murex`, read as a union)	each source loads, restates and fails on its own; each log a third the size
Domain as the first folder	matches the connector's domain; IAM and lifecycle rules can name a domain
`risk/` one table per input kind; `risk-results/` written only by the publishers	engines read `risk/`; no engine reads `risk-results/` (a guard test)
Tables by year (`trade_endur/2026/`)	DESIGNED (S2): until then one table holds every year

```
s3://acme-drishti-prod-lake/
└── lake/                    DRISHTI_DELTA_ROOT
    ├── trading/
    │   ├── trade_endur/   _delta_log/
    │   │   └── business_date=2026-10-20/
    │   │       └── PRODUCT_TYPE=IRS_FIXFLOAT/part-….parquet
    │   ├── trade_calypso/ …
    │   └── trade_murex/ …
    ├── reference/ counterparty/ market/ …
    ├── risk/                the engines' inputs
    │   ├── pnl-vector/  scenario-pv/  fx-scenario/
    │   ├── scenario-set/  risk-hierarchy/  portfolio/
    │   └── simulation-run/  exposure-path/
    ├── risk-results/        append-only, never read back
    │   ├── var-official/  exposure-official/ …
    │   └── jsonl/<table>/<date>/  (2.9.4)
    └── us-equities/ gleif/ funds/   public data
```

5.3 Partition keys and request-rate-friendly prefixes

Key	Why
`business_date` **(always first)**	every interactive read names a date; a load and a restatement replace one day; retention removes whole days; tiers by the folder's date
`PRODUCT_TYPE` **for trades (second key)**	spreads a day over several S3 prefixes; keeps each partition small enough to sort in one batch; lets a source restate one product type
nothing finer	each partition column multiplies the folders and small files of a day; two for trades, one for everything else
risk tables: `business_date` **only**	the engine reads a whole date's vectors in one streamed pass; it sorts into hierarchy order in memory

- S3 scales each prefix to ≥ 3,500 writes and 5,500 reads a second and splits busy prefixes by itself
- A day's reads spread over its product-type folders, and over three tables for trades
- Delta paths cannot be hashed (Drishti's writers name the folders): the spread comes from the partition keys
- A 503 Slow Down is retried with back-off and counted in Health; alarm `drishti-<env>-lake-s3-5xx` fires on a sustained rate

06
Keys

The id, the promoted columns, the vector columns and the statistics: what each kind of read needs from the table

6. Columns of every table

Column	Type	Content
`id`	string	the entity's id (the pack's key: `TRADE_ID`, `VECTOR_ID`, …)
`doc`	string	the whole JSON document as the source sent it (a vector replaced by a marker)
`business_date`	date	partition
partition columns	string	`PRODUCT_TYPE`
promoted columns	string or double	copies of the fields searches, lists, links and BI read: `layout.<kind>.columns`
vector columns (2.8.0)	`list<double>` (or `list<float>`)	`layout.<kind>.vectors`: `PNL`, `SCENARIO_PV`, `MTM`; one column, the length is data

- Choosing what to promote is a pack decision (DELTA_CONNECTOR.md §4); `layout.<kind>.type-ahead` adds promoted names to the type-ahead (`USEQ Ford` finds `F`)
- Adding a promoted column changes the table's columns: `lake publish` refuses a day whose columns differ → runbook (slide 94)
- Vectors are never stored twice: the document keeps `{"$vector": "PNL"}` and a fetched document gets the vector back

6. What each read needs

Need	What it reads	So the table needs
One entity (`TRD MX-20000017`)	the day's id map, then one row group of one file	rows sorted by id per file; small row groups
History (`TRD MX-20000017 30`)	the days whose id map holds it, one read per day, eight at once	`business_date` as the partition; nothing else
Type-ahead	the newest day's ids (and `index.days`), plus promoted name columns	`id`; `layout.<kind>.type-ahead` for names
Search, pick lists, derived kinds, impact, links	the day's column set: id and every promoted column, never doc	every field a search filters, sorts or shows promoted
A date's vectors (VaR, exposure)	one vector column and a few text columns, streamed file by file	`layout.<kind>.vectors`; whole dates, no deletion vectors
File skipping	per-file min/max in the log; per-row-group min/max in the footer	`delta.dataSkippingStatsColumns` = id, business_date, promoted

07
Files

Sizes, row groups, compression, statistics, checkpoints, retention, sorting by id, and one commit per day per table

7.1 The numbers

Setting	Value	Why
Rows per file	`layout.<kind>.file-rows` (250,000)	few files per day, each large enough for S3 (hundreds of MB)
Row group	1 MB (`LAKE_ROW_GROUP_KB=1024`; writer default 256 KB)	a single read fetches one row group: 1 MB balances bytes against 10–30 ms GETs
Compression	Snappy	the native engine also reads ZSTD, GZIP, LZ4_RAW
Statistics	id, business_date, promoted columns	file and row-group skipping without documents in the log
Checkpoint	every 10 commits and after every maintenance run	a reader replays the last checkpoint plus a few commits
Log retention	30 days (`delta.logRetentionDuration`); **7 years for risk**	older commit files removed by `lake maintain`
Removed files kept	7 days (`LAKE_VACUUM_HOURS` 168); **7 years for risk**	readers in flight and *known at* reach back a week (risk: 7 years)

7.2 Sorted by id

- A file holds a contiguous id range and each row group a smaller one: a read of one id skips every other row group by its statistics
- `lake build` sorts each partition (date × product type) that fits one batch (`LAKE_BATCH_ROWS`, 250,000)
- A larger partition is written in the dumps' order; load-day.sh prints a warning for each
- A table written elsewhere (Spark): `repartitionByRange(…, "id").sortWithinPartitions("id")`

DESIGNED (SUPER_SCALE S1): range maps from the log's per-file id ranges instead of id maps, and Parquet Bloom filters on id.

Three remedies, in order of preference

1. **Ask the source to export sorted by id.** The JSON Lines order becomes the file's order: sorted at any size, no memory cost (the demo's `bulk_trades.py` does so since 2.9.4).

2. **Partition finer** (`PRODUCT_TYPE`, or a book or desk column) so each partition fits a batch.

3. **Raise `LAKE_BATCH_ROWS`** with the loader's memory: rows × document size × 3; 250,000 trades of 10 KB ≈ 8 GB; the loader has 30 GB.

7.3 One commit per day per table

lake build
- writes the day into a FRESH scratch table on EFS: every file there is live

→

aws s3 sync
- copies the data files to S3: new UUID names, nothing in S3 is overwritten

→

lake publish
- ONE commit that adds the new files and removes the day's previous files

- A reader sees **the old day or the new day, never a mixture**
- The previous version stays readable (time travel, *known at*) until vacuum (`LAKE_VACUUM_HOURS`, 168 h; 7 years for risk)
- One commit per table per day ⇒ about **1,500 commits a year per table** including maintenance
- `lake publish` checks every file is in S3 before committing (on the store it commits to, since 2.7.0) and refuses a change of columns

Since 2.6.7 `lake build` and `lake demo` make one commit per business date (not per partition): sixty days of `trade_endur` went from 300 commits replaying the whole log (98 s) to 60 with a short log. Intraday risk batches are the one exception: an appended commit per batch (slide 64).

08

Data in: JSON Lines, then Delta

Since 2.9.4 JSON Lines is Drishti's data format: generate JSON Lines, then load it into Delta. CSV input is deprecated; every export offers JSON Lines

Generate JSON Lines, then load it into Delta

Source or generator
- writes JSON Lines
- `trades-<date>.jsonl.gz`

→

Landing / build folder
- `<date>/<source>/*.jsonl.gz`
- a restatement: a second file

→

lake build · load_rows.py
- promoted and vector columns
- sorted by id; one commit a date

→

Delta table
- `lake/<domain>/<table>/`
- read by the Delta connector

Form	One line is	Written by → read by
Document	one entity as its source holds it: `{"TRADE_ID": "1000015", "LEGS": [...]}`	make_trades, bulk_trades, pack samples → `lake build`, the file connector
Loader row	`{"domain", "kind", "id", "date", "doc", "columns"}`	`make_data.py --jsonl` → PostgreSQL, Cassandra, Iceberg … loaders
Lake row	`{"domain", "kind", "table", "id", "date", "doc"}`	`make_data.py --lake`, generators → `load_rows.py`
Staged lake row	`{"id", "doc", "columns"}`	lake build's staging, official-results publishers → Java `LakeWriter`

The rule: one JSON object per line, UTF-8, `.jsonl` or `.jsonl.gz`. Generating and loading are separate steps; a loader never rewrites a pack's samples. The lake built from JSONL is byte-for-byte the lake built before.

CSV in is deprecated; JSON Lines out beside every format

Coming in

- **JSON Lines first**; JSON arrays and per-entity `.json` still read
- **CSV input deprecated** (2.9.4): the file connector still serves feed files, warns once per file, names them in Admin → Health
- `drishti.py convert trades.csv` → `trades.jsonl` (also JSON arrays, gzip, folders)

Going out

- every export and download offers JSONL **beside** CSV, Excel, PDF, PNG
- server: `search/jsonl`, SQL `format=jsonl`, BI runs, statements, VaR and exposure captures
- console menus and CLI `--jsonl`; media type `application/jsonl`

```
drishti.py convert trades.csv                                    # trades.jsonl, a line per row
drishti.py convert data/feeds/fixing/SOFR-HISTORY.csv --rows-document --out data/feeds/fixing.jsonl
drishti.py convert data/feeds/settlement                         # settlement.jsonl, the file name as "id"
```

Not data, so not JSONL: Delta and Iceberg Parquet, the BI store, the VaR and exposure hot files, Arrow between the server and Python, manifests. **The guard:** two drill tests check that every generator writes JSON Lines and every export entry point offers it.

Official results: Delta, and JSON Lines beside it


```
<root>/risk-results/
├── var-official/              business_date=…/   Delta, append-only, 7 years
├── exposure-official/  exposure-official-cpty/   Delta, append-only
└── jsonl/<table>/<date>/<table>-v<table version>.jsonl      one file per commit (2.9.4)
```

Setting	Default	Effect
`drishti.var.official.jsonl`	true	each VaR publish also written as JSONL beside the table
`drishti.ccr.exposure.official.jsonl`	true	each exposure publish also written as JSONL
`drishti.var.official.root`	`${DRISHTI_DELTA_ROOT}`	where both are written; an `s3://` root is not written (publishError)

- A restatement is a new file, never a rewrite: the files follow the table's append-only versions
- Content-store row lists are JSONL too: `var/official/<date>-membership-v<n>.jsonl`, `exposure/official/<date>-trades-v<n>.jsonl`
- Records written before 2.9.4 stay `.json` and are read as they are: the store is append-only

09
Loading

The daily load, the data-load signal, why concurrent commits are safe, backfill and restatement

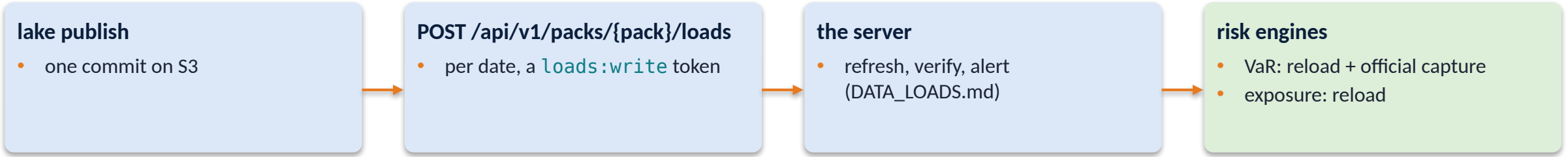
9.1 The daily load (load-day.sh)


```
EventBridge Scheduler (LOAD_SCHEDULE, LOAD_SCHEDULE_ZONE)
  └─► ECS RunTask drishti-<env>-loader ─► loader-entrypoint.sh (EFS mounted? writable?) ─► load-day.sh
        for each table of LAKE_TABLES_FILE:
          flock  /opt/drishti/data/lake-locks/<domain>--<table>.lock   (held: exit 3)
          aws s3 sync   s3://landing/<date>/<source>/  →  scratch/in/<table>/      (JSON Lines)
          drishti.py lake build   --from scratch/in/<table> -r --kind trade --pack config/packs/trading
                                  --mode overwrite-dates --batch-rows 250000 --partition-by PRODUCT_TYPE …
          aws s3 sync   scratch/lake/trading/trade_endur/ → s3://lake/…/trade_endur/  --exclude "_delta_log/*"
          drishti.py lake publish --from scratch/lake/… --to s3://…/trade_endur --dates <date> [--signal …]
          CloudWatch Drishti/Lake: LoadSucceeded, PublishedBusinessDate, LoadSeconds
        scratch removed (always)
```

Exit	Meaning
0	every table published
2	published with rejected documents (the summary says which)
1	a table failed: its S3 table is unchanged
3	a table was locked

The date defaults to today in `LOAD_SCHEDULE_ZONE`, the zone the schedule runs in: an evening load in New York is already the next day in UTC.

9.1 The data-load signal



- With `DRISHTI_SIGNAL_URL` (a node's Cloud Map name) and `DRISHTI_TOKEN_SECRET_ID` (an API token with `loads:write`, read into a 0600 file)
- Without it, servers see the new table version at their next refresh (`refresh-seconds`, 10)
- Either way readers switch at the commit; the signal makes it immediate and runs the load's expectations and alerts
- **Risk kinds (2.8.0):** a `pnl-vector`, `scenario-pv`, `scenario-set`, `risk-hierarchy` or `portfolio` signal reloads the VaR date and captures it officially; an intraday batch is applied on top; an `exposure-path` signal reloads exposure

> Signals are safe in bursts since 2.6.7: reads queue at most `max-queued-reads` (4,096) and a read whose caller has gone leaves the queue.

9.2 Why concurrent commits are safe

Writer	How it commits
`lake publish`, `lake maintain` (Python deltalake 1.x)	conditional PUT of the commit file (`If-None-Match: *`): the second writer gets 412, re-reads the log and retries (20); no DynamoDB lock; unsafe rename refused
Drishti's Java writer (`lake build`, the official-results publishers, Kafka intraday batches)	local disk and EFS only (a hard link of a finished commit file); never S3
Drishti's native Iceberg writer (drishti-lakeio)	the same conditional PUT, with a start-up probe
The content store's s3 backend	`If-None-Match: *` to create, `If-Match: <ETag>` to update; the loser gets `409 DRS-5057`

Two layers of safety. `check-conditional-writes.sh` proves the bucket refuses a second create. On top, the scripts take a lock per table on EFS (flock, NFSv4 byte-range locks, released if a task dies), so a load, a backfill and maintenance of one table never overlap. **Tested:** three `lake publish` runs at once on moto, one winner per version.

9.3 Backfill and 9.4 restatement


```
deploy/aws/delta-lake/backfill.sh --from 2026-01-02 --to 2026-10-16 --skip-missing          # weekdays; dates without dumps skipped
deploy/aws/delta-lake/backfill.sh --from 2025-01-02 --to 2025-12-31 --from-dir /opt/drishti/data/history
```

- **Backfill:** oldest date first, one load-day.sh per date
- Stops at the first date that fails and prints the command that resumes from there
- Re-running a date replaces it: a backfill can always be run again
- Large backfill: several loader tasks side by side on different tables (`--tables`); the per-table lock keeps two apart

A source re-sent	Run
some whole days	`backfill.sh --dates 2026-10-14,2026-10-15`
one product type of a day	`load-day.sh --date 2026-10-15 --tables trading/trade_endur --mode overwrite-partitions`

Restatement: readers switch at the commit; *known at* before it still shows the day as it was, for `LAKE_VACUUM_HOURS` (7 years for risk). A restated old day's files move to its tier at the next lifecycle run. A restated VaR or exposure date gets the next `RESULT_VERSION` in the official results; the earlier one is kept.

10

S3: safe commits and three simulators

The content store on S3, the conditional writes each piece relies on, and how they are tested without an AWS account (2.7.0)

The content store: cluster state on S3 without a shared disk

One interface, four backends (config/store/store.yaml) BUILT

- get · compare-and-set put · list · delete · changes

filesystem

- default; a shared mount (EFS) on a cluster

sqlite

- single node

postgres

- one SQL table

s3

- conditional PUT; ETag versions

- **s3:** `<prefix>/items/<key>`; create `If-None-Match: *`; update `If-Match: <ETag>`; a lost race is `409 DRS-5057`
- Two nodes saving one item at once: one success, one `409`; an unreachable store `503 DRS-5059`
- A dead node's BI run is adopted by exactly one node, by a compare-and-set lease

What it holds on a cluster

- Designs; BI workspaces, items, versions, runs, schedules
- data-load overrides, connector edits, collaboration
- VaR and exposure captures and official records (append-only; row lists JSONL since 2.9.4)

On ECS

`s3` or `postgres` need no shared disk; `filesystem` needs EFS at the same path on every node. The s3 bucket needs the conditional-write check of step 4.

Three S3 simulators, no Docker, and what each covers

Simulator	What it tests	Runs on	Conditional writes
Adobe S3Mock 4.12.4 (in the test JVM)	content store on S3; Iceberg native engine's table on S3	Linux, Windows	create-only, update-if-match, delete-if-match; racing creates **not** atomic
moto 5.2.3 (`moto[server]`, pure Python)	Delta on S3 via deltalake; `lake publish`: new table, a day restated, a missing file refused, three publishes at once	Linux, Windows	all three; racing creates have one winner
MinIO server binary	all of the above, the most faithful	Linux only (binary needed)	all; skipped with the reason when no binary

```
./mvnw -q -pl drishti-identity,plugins/drishti-plugin-iceberg -am test -Dtest='S3MockContentStoreTest,NativeOnS3SimulatorTest' …
drishti-console/.venv/bin/python -m unittest -v tools/lake/test_s3_simulators.py      # moto: Delta and lake publish
```

> Fixed in 2.7.0: `lake publish --storage-option AWS_ENDPOINT_URL=…` checks the files on that endpoint, as the commit does.

11
Maintenance

Checkpoints, retention and the risk lake's 7-year floor, vacuum, compaction now that relayout knows the per-source tables, the lifecycle rules, and Admin → Lakes

11.1 What maintain.sh runs (nightly)

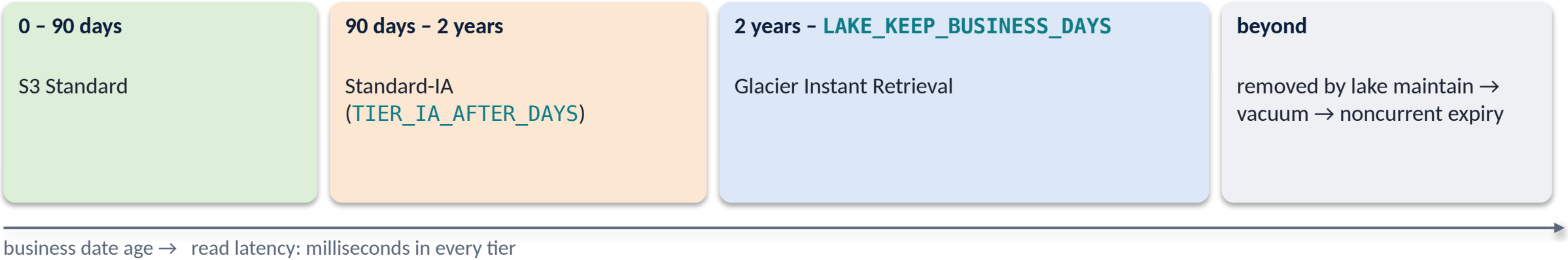

```
drishti.py lake maintain --config <lake-maintenance.s3.yaml rendered> --once --force-foreign --tables trading/trade_endur
```

Nightly (`MAINTAIN_SCHEDULE`, Tuesday to Saturday 02:30) as the maintenance task, per table under the table's lock; then setup-lifecycle.sh.

Key	Value	What lake maintain does
`keep-business-days`	`LAKE_KEEP_BUSINESS_DAYS` (1800)	removes dates older than that, counted back from today: one commit; refuses to remove more than `max-drop-share` (0.5)
`checkpoint`	true	writes a checkpoint and removes commit files past `delta.logRetentionDuration`
`vacuum-hours`	`LAKE_VACUUM_HOURS` (168)	deletes files no version uses, older than a week; S3 keeps them 14 more days as noncurrent versions
`compact`	false	see 11.3
risk floor (2.8.0)	7 years: 1,827 days, 61,368 h	`risk/` and `risk-results/` never below it, whatever the settings (`retention-floor`)

> `--force-foreign`: the S3 tables are Drishti's (lake publish wrote them), so the script overrides the ownership rule and marks the runs forced.

11.2 Retention and the tiers together



- **Never** Glacier Flexible Retrieval or Deep Archive: a restore of hours before a read; Drishti reads any day in milliseconds
- **Never** an S3 expiry of current lake objects: removing a file a table version still names breaks the table
- Only `lake maintain` removes table files; S3 then keeps the noncurrent versions 14 days

The risk lake (2.8.0): `risk/` and `risk-results/` keep 7 years of dates and time travel. Their old months sit in Glacier IR, still read in milliseconds, so any official VaR or exposure figure can be recomputed from its inputs years later.

11.3 Compaction: off for S3, and now safe to turn on

- Each day arrives laid out: files of `file-rows`, sorted by id, one commit ⇒ the S3 tables need no compaction
- A plain Delta compaction would merge files out of id order (single-entity reads would touch many row groups)
- `lake maintain` re-sorts (**relayout**) instead of compacting where the pack declares a layout
- So `compact: false` stays the shipped S3 setting; a day that needs re-laying is restated (slide 94)

Fixed in 2.6.7

`lake maintain` finds the layout of the connector whose table it is, so `trade_endur`, `trade_calypso` and `trade_murex` are re-laid out by id, not compacted out of order; relayout keeps `business_date` and `PRODUCT_TYPE` and checks each.

Turn compaction on for S3 after a relayout test there

PROPOSED

- the first edition's open finding is closed; AWS_DELTA_LAKE.md 9.3 and `lake-maintenance.s3.yaml` still describe it as open
- run `lake maintain --dry-run` with `compact: true` on one S3 table, check id order per row group, then enable it nightly

11.4 The lifecycle rules (setup-lifecycle.sh)


```
{"ID": "tier-trading-trade_endur-2026-03", "Status": "Enabled",
 "Filter": {"And": {"Prefix": "lake/trading/trade_endur/business_date=2026-03",
                    "ObjectSizeGreaterThan": 131072}},
 "Transitions": [{"Date": "2026-06-29T00:00:00Z", "StorageClass": "STANDARD_IA"},
                 {"Date": "2028-03-30T00:00:00Z", "StorageClass": "GLACIER_IR"}]}
```

- Rules match prefixes and move on a date or an age; age is wrong for restated days ⇒ one rule per table per business month, with dates
- A month's files move to Standard-IA when its last day is 90 days old, and to Glacier IR at two years
- Rules exist three months ahead; maintain.sh rewrites them nightly
- `_delta_log/` never matches: commits and checkpoints stay in Standard; objects under 128 KB stay where they are
- S3 allows 1,000 rules; the script refuses more (≈ 30 tables over seven years)

Costs to know: Standard-IA keeps an object at least 30 days, Glacier IR 90 days; a restatement of a tiered day pays the rest of that minimum. Reads from Standard-IA ≈ USD 0.01/GB, from Glacier IR ≈ USD 0.03/GB.

11.5 Admin → Lakes: the same engine in the console

Step	What it does	Setting
Retention	deletes business dates older than the window, counted from today; refuses to drop more than half a table	`keep-business-days`
Compaction	merges small files, or re-lays a table in its pack's layout (dates that drifted only)	`target-file-mb` (128)
Checkpoint	readers replay a short log	`checkpoint` (on)
Vacuum	removes files no version uses, older than the window	`vacuum-hours` (168)

- **Dry run** first: files and size before → after, dates retention would drop, what vacuum would delete
- **Ownership rule:** only tables `lake build` built; a foreign table is read-only
- Schedules at a time or after every build; history kept in `maintenance-history.jsonl`

On AWS: one scheduler

EventBridge runs `maintain.sh` nightly against `s3://` with `--force-foreign`.

Admin → Lakes maintains the lakes listed in `lakes.roots` and the file `<data>/lakes/lake-maintenance.yaml`; dry runs and history work the same.

Run schedules in **one** place: set `DRISHTI_LAKE_MAINTENANCE_SCHEDULES=false` on the console when EventBridge or the daemon runs them.

The risk floor applies in every place it runs.

12

The risk lake: VaR and exposure

Inputs as vectors in Delta, computed for any scope when asked, official results published append-only with JSON Lines beside them; morning runs, intraday batches and Kafka micro-batches

The risk lake's tables: inputs in risk/, results in risk-results/

Table	One row is	Key	Vector column	Read by
risk/pnl-vector	a trade's (or position's) P&L over 509 scenarios	VECTOR_ID	PNL	VaR engine
risk/scenario-pv (2.9.4)	a trade's scenario PVs and base PV, its own currency	SCENARIO_PV_ID	SCENARIO_PV	VaR engine
risk/fx-scenario (2.9.4)	a currency's rate today and per scenario	FX_SCENARIO_ID	none (RATES in doc)	VaR engine
risk/scenario-set · **risk-hierarchy** · **portfolio**	the scenario days · a tree node with its limit · a portfolio	SCENARIO_SET_ID …	none	VaR engine
risk/simulation-run	an exposure run's grid and identity	RUN_ID	none	exposure engine
risk/exposure-path	a trade × grid date: MtM over the P paths	PATH_ROW_ID	MTM	exposure engine
risk-results/var-official	a node × date × result version	node@date#v	none	BI only (VOFF)
risk-results/exposure-official(-cpty)	a node, set, counterparty or group × version	scope@date#v	none (profile as JSON)	BI only (XOFC)

> Engines read risk/ only; nothing reads risk-results/ back except BI's read-only connectors (a guard test). Positions are derived, never stored.

A vector is one list<double> column: why, and how it is carried

Layout (one date, 2M rows × 509)	Size, compressed (estimate)	Sum a scope's vectors	Verdict
A. Long: a row per position × scenario	~9–11 GB	a group-by over hundreds of millions of rows	rejected
B. Wide: 509 columns	~5.5–6.5 GB	509 column chunks per row group	rejected (schema = scenario count)
C. Vector: `PNL list<double>`	~5.5–6.5 GB	one column, decoded into one array, added	**chosen, built**
D. Blob: 4,072 bytes	~6–7 GB	one column, copy bytes	rejected (other tools cannot read it)

As built (2.8.0, exposure 2.9.0)

BUILT

- `layout.<kind>.vectors: [PNL]`; `lake build --vector`; both writers
- the document keeps `{"$vector": "PNL"}`: never stored twice
- streamed per date, one buffer reused for every row
- round trip JSONL → Delta → `double[]` equal to the bit
- `list<float>` read and written for exposure paths

Where it runs

- streamed reads: the **Delta plugin only** (`SourcePlugin.vectors`)
- VaR and exposure inputs on Iceberg: not built (companion deck: PROPOSED)
- bank-scale layout (sort by hierarchy, ZSTD + byte-stream split): DESIGNED

Morning run, intraday batches, Kafka micro-batches: all into the lake

Morning run · RUN_KIND FULL

- every live trade, one BATCH_ID
- JSONL → `lake build` replaces the date

Intraday files · INTRADAY

- changed trades only, own BATCH_ID
- `drishti.py var intraday --load` (append)

Kafka PV topic · micro-batches

- `pv-updates` (var-pv-stream), JSONL text
- window 5 min · flush at 500 · Apply now

risk/ in Delta (business_date=D)

- the morning commit replaces D
- each intraday batch: one appended commit
- each Kafka window: ONE batch, `K-<date>-<time>-<n>`
- latest `COMPUTED_AT` per trade wins
- the lake is the record: a restart rebuilds
- 7 years of time travel

signal

VaR engine (one date hot)

- full run: date reloads
- batch: new − old into the book and every node above
- official capture from the morning run only

risk-results/var-official

- append-only Delta, a row per node and version
- JSONL beside it (2.9.4)
- read by BI as `VOFF`

BUILT Measured: 200 batches of 3 trades equal one full load to the cent, on every node; batch time does not grow.

On AWS: who writes the risk lake, and where

Table group	Written by	On an S3 lake today
`risk/` morning runs (files)	the bank's batch: JSONL → `lake build` → `lake publish` (overwrite-dates)	BUILT: load-day.sh, like any table
`risk/` intraday files	`lake build --mode append` → `lake publish --mode append`	BUILT tools; no script yet
`risk/` Kafka windows	the server's Java lake writer (`pv-updates' lake`)	local or EFS root only
`risk-results/` official	the server's publishers (Java lake writer) + JSONL	local or EFS root only; `s3://` → `publishError`

Risk tables in S3 keep 7 years: publish them with `--log-retention-days 2557 --deleted-file-retention-days 2557`; `lake publish`'s defaults (30 and 7 days) would cut time travel.

Official results to S3

PROPOSED

- `drishti.var.official.root` on EFS (and the exposure root)
- after each capture: `lake publish --mode append` to `s3://…/risk-results/…`; copy the JSONL files
- BI's `var-results` reads the S3 copy

Kafka windows to S3

PROPOSED

- `pv-updates' lake` on EFS; each window published with `--mode append`
- or keep `risk/` on EFS if intraday must be seconds

Hot files: `DRISHTI_VAR_HOT_DIR`, `DRISHTI_EXPOSURE_HOT_DIR` on fast local disk, as the guides say (section 14).

Official results: append-only, versioned, reproducible

Column group	Content
`business_date` · `id` · `doc`	partition; `<NODE_ID>@<date>#v<RESULT_VERSION>`; the row as JSON
node	`HIERARCHY`, `NODE_ID`, `NODE_NAME`, `PARENT_ID`, `LEVEL`
version and inputs	`RESULT_VERSION` (1, 2 … on restatement), `CAPTURE_ID`, `INPUT_VERSIONS` (versions, batch ids, rules digest)
figures	`VAR`, `ES`, `CONFIDENCE`, `ES_CONFIDENCE`, `STANDALONE_SUM`, `DIVERSIFICATION`, `HYPO_PNL`
limits and backtest	`LIMIT`, `LIMIT_USE`, `EXCEPTIONS_250`, `TRAFFIC_LIGHT`, `TRADE_COUNT`, `POSITION_COUNT`

- `delta.appendOnly`: nothing is overwritten; a restated date adds rows with the next `RESULT_VERSION`
- 7 years of log and removed files: time travel to any published day
- no vectors, no trade rows: the capture in the content store keeps the vector for audit
- exposure: `exposure-official` and `-cpty`, the profile as JSON text

For other systems, never for engines. BI reads the published tables as `VOFF` and `XOFC` (the reports Market risk: VaR and Counterparty risk; no measure sums VaR). Each figure can be recomputed from its inputs, the same vectors and engine version, years later.

Exposure paths: the large table, and what is kept for 7 years

Item (1,000,000 trades)	P = 2,000, G = 50	P = 5,000, G = 100
trade paths, float32, dense, a date	400 GB	2.0 TB
the same, float64	800 GB	4.0 TB
netting-set V matrices (hot), 100,000 sets	40 GB	200 GB
for comparison: VaR vectors, a date	4.07 GB	

What	Kept (decided 2026-10-26, built 2.9.0)
trade paths	20 business days, and each month-end for 1 year
netting-set matrices	every date for 1 year; month-ends for 7 years
statistics per set and counterparty	every date, 7 years
trade EE contributions	every date, 7 years
official captures, run records	7 years

3.5 PB → about 150 TB: 7 years of dense trade paths at P = 5,000, G = 100 would be 3.5 PB; under the decided tiers the standing storage is about 150 TB at full size (a fifth at P = 2,000, G = 50). ESTIMATED from the guide's formulas.

Exposure-path layout at scale (EXPOSURE_GUIDE.md 15.3)

PROPOSED

- partition by `business_date` and `run_id`; sort by `NETTING_SET`, `TRADE_ID`, `GRID_INDEX`; ~512 MB files; `MTM` BYTE_STREAM_SPLIT + ZSTD

13

Bank scale: 2.3 million positions, 15,000 portfolios

What a bank of that size puts in the lake each day, where Delta matters, what is measured and what is estimated

What 2.3 million positions put in the lake each day

Per business date	Arithmetic	Result	Basis
P&L vectors, float64 (in the lake)	2.3M × 509 × 8 B	9.37 GB	arithmetic
... float32 (the hot copy)	2.3M × 509 × 4 B	4.68 GB	arithmetic
... as Parquet (≈ 0.75 × raw)	9.37 GB × 0.75	≈ 7.0 GB	ESTIMATED
... a year; 7 years kept	× 252; × 7	≈ 1.77 TB; ≈ 12.4 TB	ESTIMATED
Partitions; files at ~512 MB	1 (business_date); 7.03 GB ÷ 512 MB	1; ≈ 14 files	DESIGNED
A cold read of the date	7.03 GB ÷ 1 MB row groups	≈ 7,000 GETs	arithmetic
Portfolio definitions	15,000 rows a date	small	arithmetic
Every portfolio's vector, if stored	15k × 509 × 8 B	≈ 61 MB per currency	PROPOSED
Kafka micro-batches	5-minute window	≤ 12 commits an hour	arithmetic
Intraday records on the topic	10,000 updates × 4–8 KB	40–80 MB a day	guide

The risk lake is small next to the documents

≈ 12.4 TB of VaR vectors over 7 years, against ≈ 200 TB of documents (slide 88). Exposure paths are the large table (slide 67).

Position grain (2.9.4)

A system may send one row per position: rows = positions, so 2.3M rows a date whatever the trade count. 15,000 portfolios are far under the 100,000-row read limit.

Where Delta matters at bank scale

Partition by business_date — BUILT

a full run replaces its date; intraday batches append

Vector column list<double> — BUILT

standard Parquet; a sum reads one column

Time travel 7 years — BUILT

a restatement is a new commit; reproduce any official number

Checkpoints + lake maintain — BUILT

many small intraday commits stay fast; 7-year floor

Official results append-only — BUILT

Delta + JSONL beside; BI reads them

Delta read from S3 — BUILT

ranged GETs; one row group; footers cached

Sort by LE, desk, book; ZSTD + BSS — DESIGNED

a node is a contiguous run of rows

VaR sharding across servers — DESIGNED

each server maps its ranges; partial vectors added

Z-order / liquid clustering — PROPOSED

skip files for a desk's or portfolio's read

Node and portfolio vector table — PROPOSED

BI and peers read sums, not 2.3M rows

Change data feed on inputs — PROPOSED

followers see intraday deltas

Official results to S3 — PROPOSED

EFS root, then `lake publish --mode append`

Measured on the documents: a million trades a day, seven years

Question (1,000,000 trades a day)	Measured
type-ahead `TRD CLY-40834`	29 ms
open a trade, first time / cached	137–157 ms / 48 ms
open a trade on a past business day	288 ms
a search over the day, exact over 1,000,000	134–322 ms
desk P&L of all desks / another desk	327 ms / 48 ms
impact of a netting set (143,982 trades)	716–881 ms
a search on another business day, first / again	6.6 s / 306 ms
server heap in use	2–3.5 GB

Seven years: 1,800 days, 7,200 files	Measured
start-up incl. file list and newest ids	729 ms
a trade on 2019-12-05	22 ms
a trade on 2022-06-15	13 ms
the file list re-read	84 ms
an old day's columns	13 ms

MEASURED

Nothing in the read paths grows with history: a read concerns one business day, the file list comes from the last checkpoint, caches hold days by memory. A day in 2019 waits once for its id map (about a second).

Reading one VaR date at bank scale: lake, hot file, memory

The lake (S3, Parquet)

ESTIMATED

≈ 7.0 GB a date; a cold whole-bank pass ≈ 18–70 s on one server

The hot file (local disk)

ESTIMATED

≈ 5.5 GB a date; reopened in ≈ 0.5 s; 60 dates ≈ 328 GB

Memory (page cache + heap)

ESTIMATED

heap ≈ 1.8 GB a date; whole bank ≈ 78 ms resident, ≈ 1.6 s cold from NVMe

× 23 = scaled from the measured 100,000-trade points (VAR_GUIDE.md 27.3); a node is size-independent. The 1 M × 50 run on a real server is still to do; VaR sharding (DESIGNED) is the step for a date larger than one server.

Measured at 100,000 trades	Time	× 23
first load of a date	0.8–1.2 s	≈ 18–28 s
reopen from its hot file	23 ms	≈ 0.53 s
a node's VaR and ES	0.48 ms	≈ 0.5 ms
the whole bank, float32	3.4 ms	≈ 78 ms

On ECS

`DRISHTI_VAR_HOT_DIR` and `DRISHTI_EXPOSURE_HOT_DIR` on fast local disk (NVMe), as the guides say: the default `<data dir>` is on EFS. Hot files are rebuildable from the lake, so they meet the NVMe exception of slide 75.

14

Caches: EFS or local NVMe

The small, bounded structures each server keeps, where they live, and the one decision for the product owner, now with the VaR and exposure hot files

14. What each server caches

Cache	Where	Bounded by	After a restart
file list (snapshot) of each table	heap	the table's live files	re-read from the checkpoint
Parquet footers	heap (50,000 row groups)	row groups	re-read on first use (one ranged GET)
id maps of days	heap (20 % shared) and disk (`id-map-disk.dir`)	memory; a file per version and date	memory-mapped from disk: no scan
column sets of days	heap (`columns-cache-mb`)	memory	re-read (newest day in the background)
documents	heap and RocksDB on disk (`doc-disk-cache.dir`, 1 GB, 12 h)	size, age	warm from disk
history answers (2.6.5)	heap (`drishti.history.cache-mb`, 256 MB)	memory; by the kind's data version	rebuilt on ask
VaR hot files (2.8.0)	`DRISHTI_VAR_HOT_DIR`: fast local disk	2 dates open, 60 on disk	rebuilt from the lake (deleted at shutdown by default)
exposure hot files (2.9.0)	`DRISHTI_EXPOSURE_HOT_DIR`: local disk	dates within retention	rebuilt from the lake

> The disk caches default to `<data dir>/cache/…`, on ECS on EFS; the hot files default under `<data dir>` too.

14. EFS or local NVMe for the disk caches and hot files

	EFS (default)	Local NVMe (instance store)
Latency	≈ 1 ms an operation (NFS); page cache serves repeated reads	tens of microseconds
After a task moves host or zone	warm: same folders	empty: rebuilt as users read
Launch type	Fargate or EC2	EC2 only (m6id, r6id, i4i)
Cost	EFS Standard ≈ USD 0.30/GB-month + Elastic charges	included in the instance
Identical-path rule	satisfied	not: the cache path is on the host
Memory-mapped VaR and exposure hot files	works, but each page fault is an NFS read	what the guides ask for: fast local disk

```
# config/connectors/trade-endur.yaml
id-map-disk.dir:
  /opt/drishti/nvme/cache/trade-endur/idmaps
doc-disk-cache.dir:
  /opt/drishti/nvme/cache/trade-endur/docs
# the server task
DRISHTI_VAR_HOT_DIR=/opt/drishti/nvme/var/hot
DRISHTI_EXPOSURE_HOT_DIR=/opt/drishti/nvme/ccr/hot
```

Recommendation

Start the connector caches on EFS. Put the VaR and exposure hot files on NVMe from the start (`register.sh --nodes --cache nvme`, EC2): they are rebuildable and memory-mapped. The NVMe exception is the product owner's decision.

15

Staying fast past 200 TB

What never scans the whole lake, history over the lake, spreading the work over Drishti*net, the numbers to watch and the alarms

15.1 What never scans the whole lake

Request	Reads	Grows with history?
Open one entity on a date	the snapshot (in memory, refreshed every 10 s), the day's id map, one row group (≈ 1 MB)	no
History of one entity over N days	the days whose id map holds it; one read per day, eight at once	with N, not with the lake
Type-ahead	the newest day's ids and promoted names in memory	no
Search, pick list, derived kind, impact, DATAMAP count	that day's column set (id + promoted, never documents)	no
A VaR or exposure date	one streamed pass over the date's vectors, then the hot file	no
Load, restatement, intraday batch	one day's files and one commit	no
Retention	a delete by partition (the log only), one commit	no
Vacuum (weekly is enough)	a listing of the table's prefix	with the file count
Questions over all of history	not interactive: Athena, Trino or Spark on the same tables	—

15.2 History over the lake: TRD <id> 30


```
TRD 1000015 30                    # the trade over its last 30 business days with data
TRD 1000015 3M DIFF               # three months, each day's changes
GET /api/v1/history/trade/1000015?days=30&knownAt=2026-10-15T18:00:00Z    # as the lake stood then
```

days()

- the snapshot's `business_date=` partitions whose id map holds the id

one read a day

- id map → one row group; eight days at once

mask, then diff

- each day masked for the caller; changed fields

answer, cached

- by the kind's data version

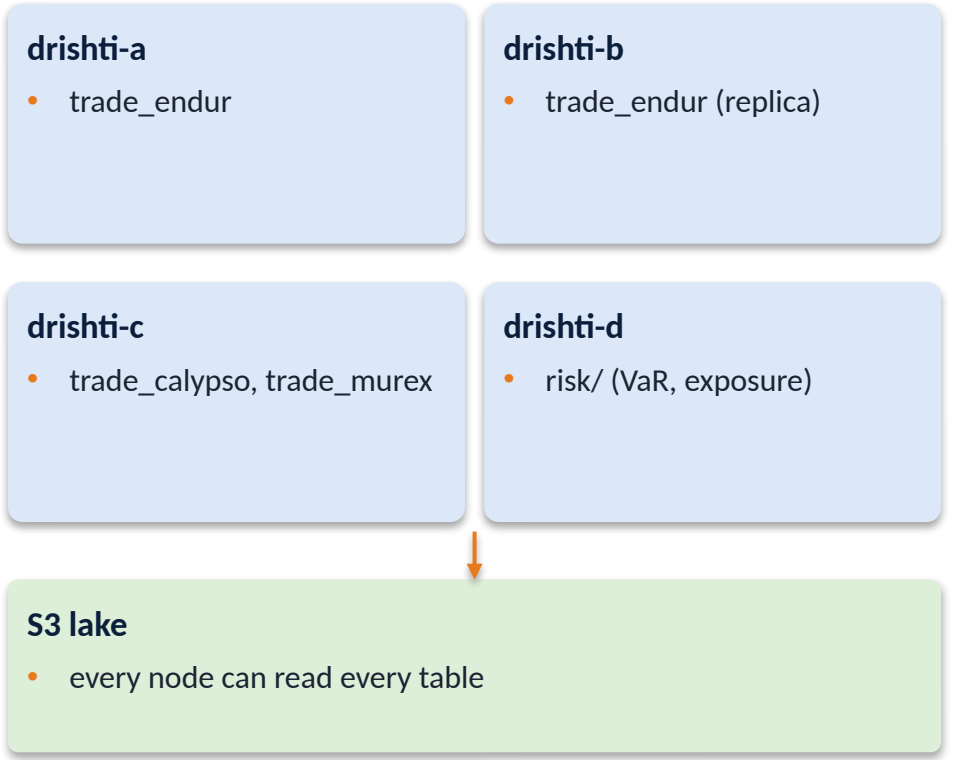
- A weekend, a holiday or a day the entity was not loaded costs nothing
- `knownAt` is Delta time travel: the day as the table stood then
- Across Drishti*net: one call to the peer that holds the data, masked on both sides
- Works the same on Iceberg (equality test) and on every connector (the contract's default)

Measured on 1,800 days

a trade on 2019-12-05 in 22 ms; the history reads only the days that hold the entity, so 7 years of lake cost no more than the days asked.

15.3 Spreading the work over Drishti*net

- **Shards:** nodes whose connectors read different tables of a kind; a search asks every one; a read goes to the holder
- **Replicas:** nodes on the same table; a forwarded read goes to the same node for the same entity (hash of its id)
- Every node can read every table from S3: a failed node costs speed, never answers
- **Since 2.6.7:** peer data over HTTP/2, one connection per peer; gossip on its own port; history across nodes
- **VaR:** each server loads whole dates; sharding one date is DESIGNED



DESIGNED (SUPER_SCALE S5, S6; VAR_GUIDE.md 28):

ownership of (table, day) slices on the ring, so N servers on one table hold N × one server's working set; and a VaR date sharded by legal entity and desk, partial vectors added in a fixed order.

15.4 The numbers to watch: report.sh


```
bucket acme-drishti-prod-lake (CloudWatch daily storage metrics):
  StandardStorage                        10412.3 GB
  StandardIAStorage                      46120.8 GB
  GlacierInstantRetrievalStorage        143998.1 GB
  objects                          672310

table                       version  checkpoint  behind   dates       newest   files GB(newest)    <32MB
trading/trade_endur            4512        4510       2    1764   2026-10-20     128      41.37        0
```

- `report.sh` is scheduled daily (`REPORT_SCHEDULE`), prints these and sends them to CloudWatch; numbers illustrative for ≈ 200 TB
- **behind** = commits since the last checkpoint: intraday risk tables add commits all day; checkpoints every 10 keep it short
- **<32MB** = small files of the newest day: should be 0 for the documents; intraday batches are small by nature (maintenance compacts them)

15.4 The numbers to watch: healthy values and what to do

Number	Healthy	If not
behind (commits since the checkpoint)	under 10	a checkpoint failed: run maintain.sh; alarm log-long above 50
files of the newest day, and <32MB	≈ rows ÷ file-rows per partition; 0 small	tiny dumps or partitions too fine (5.3)
live files per table	under ≈ 100,000	split the table (by source, or by year: S2, DESIGNED)
S3 5xx	rare	503 Slow Down on one prefix: spread with partitions; alarm lake-s3-5xx
Admin → Health p50Ms, p99Ms, cold reads	p95 < 300 ms newest days, < 1 s any day	footers and id maps not warm, or EFS latency
Admin → Caches: the one memory budget	under its cap	raise the heap (8 GB per million entities a day) or shard
Admin → Health `gossipPort`, `peerProtocol`	a port, `http2`	0: gossip on the server port (a firewall; slide 27)
LoadSeconds	well inside the window	one loader task per table, or larger loader tasks

15.5 The CloudWatch alarms (setup-alarms.sh)

Alarm	Fires when
`drishti-<env>-<table>-load-failed`	a load of the table failed (nothing of it was published)
`drishti-<env>-<table>-not-loaded`	no successful load for three days running (a weekend is two)
`drishti-<env>-<table>-load-slow`	a load took longer than `ALARM_PUBLISH_DEADLINE_HOURS`
`drishti-<env>-<table>-log-long`	readers replay more than `ALARM_COMMITS_SINCE_CHECKPOINT` commits
`drishti-<env>-lake-s3-5xx`	more than `ALARM_S3_5XX_PER_5MIN` server errors in 5 minutes, twice in 15
`drishti-<env>-efs-io-limit`	EFS PercentIOLimit above 80 %
`drishti-<env>-<node>-down`	a node's service has no running task for 5 minutes

All notify `ALARM_SNS_TOPIC_ARN`. Heap, Drishti*net member dead and backup failed come with the platform's agent; gossip latency is the metric `drishti_net_gossip_exchange_seconds` (2.6.7).

16

The demo lake, public data and DATAMAP

84 Delta tables from committed JSON Lines, the public-data packs on the same pattern, and the data map that checks every join against the lake

The demo lake: 84 Delta tables from committed JSON Lines

What	Value
Tables	**84** (2.9.4), about 2.7 GB
Business days	60, ending today's business date
Built in	943 s (about 15 minutes), measured
From	committed `data/testdata/…/*.jsonl.gz` (about 120 MB)
Trades	15,000 on the last day in 3 tables (ENDUR, CALYPSO, MUREX)
Risk	VaR inputs, scenario PVs, FX scenarios, 73 netting sets of exposure paths

```
drishti.py lake demo                          # all 84 tables
build_demo_lakes.py --store calypso=iceberg
build_demo_lakes.py --names config/demo/names.yaml
```

- **The pattern:** `data/testdata/<domain>/<kind>/*.jsonl.gz` → `build_demo_lakes` → Delta → the pack's Delta connector
- tested with the demo connector **off**: a build test fails any shipped kind without a Delta connector
- the same commands build a lake on EFS for a proof of concept

Domains: trading, reference, legal, collateral, credit, **risk**, market, macro, news, civic, us-equities, gleif, funds. **Not a production ETL:** in production your own loads end with the data-load signal (DATA_LOADS.md), and on AWS the loader task of section 9 does the work.

Public-data packs: real data on the same lake pattern

us-equities

- 479 companies with SEC filings
- filings, facts, index members, SIC
- prices internal only, off

gleif

- ≈ 9,500 legal entities
- 540 ownership groups, 132 countries
- LEI, BIC, ISIN mappings

funds

- 200 largest US funds and ETFs
- N-PORT holdings, N-CEN facts
- index weights from SEC data

refresh script

- `tools/publicdata/<pack>.py refresh`

→

committed snapshot

- `snapshot-<date>.jsonl.gz`

→

lake demo

- `<pack>/<kind>` every demo day

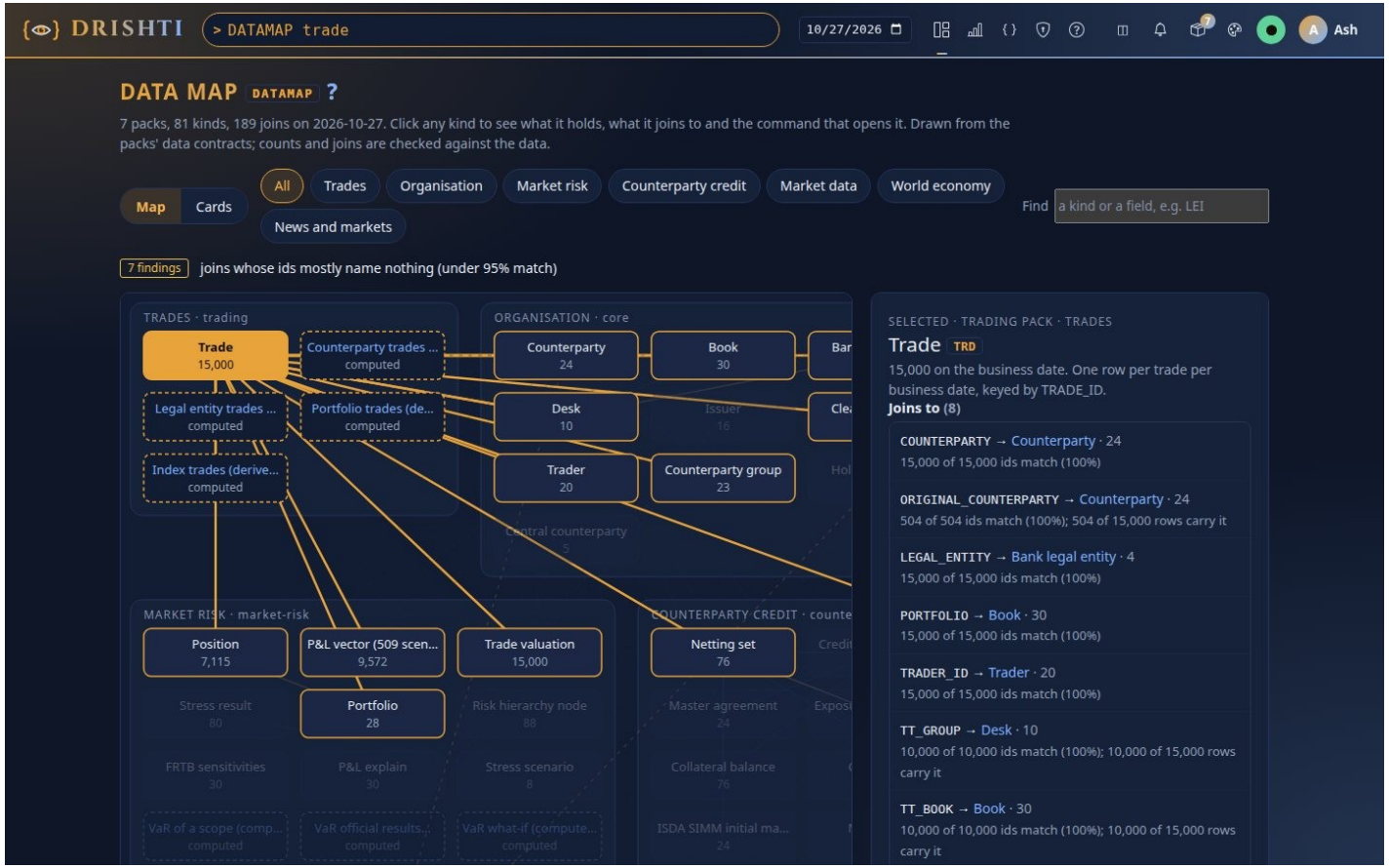
→

Delta connector

- `us-equities-store`, `gleif-store` …

- One standalone refresh script per pack (`status`, `refresh`, `--dry-run`, `--offline`), `.sh` and `.ps1` wrappers
- Internal-only sources (IMF, Yahoo, news) are refused in committed folders without `--internal-ok`
- Names in the type-ahead from promoted columns: `USEQ Ford` finds `F` (`layout.<kind>.type-ahead`)

DATAMAP: every kind, count and join, checked against the lake



`DATAMAP trade`: the Trade kind (15,000) and its joins, each checked against the lake (USER_GUIDE.md, 2026-10-27).

- kinds from the packs' contracts; computed kinds (VaR, exposure) dashed
- joins from `links:`, `link:` and `graph.fields`, **each checked against the lake**: under 95 % match is a finding
- counts per business date, cached 15 minutes; read from the day's columns, never documents

On the demo lake

10 packs: 96 kinds, 226 joins, 176 checked in about 1.5 s; 11 findings to fix in the contracts or the data.

17

Sizing and cost

What 200 TB of documents is, the deployment that serves it, and the monthly bill (list prices, one Region, estimates); the risk lake's share from section 13

17.1 What 200 TB is (seven years of Parquet)

	Value
Per business day (252 a year)	about 113 GB of Parquet; about 100 million documents at about 1.1 KB each compressed (Snappy)
Files per day	about 350 (330 MB each, 250,000 rows) over all tables
Live files after seven years	about 620,000 over all tables; three trade tables ≈ 200,000 each: above 100,000, so tables by year (S2) or more tables matter before year seven
Commits	one per table per day, plus maintenance: about 1,500 a year per table (intraday risk tables more)
The risk lake at 2.3M positions	≈ 7 GB of VaR vectors (Parquet) a date, ≈ 12.4 TB over 7 years (section 13)

ESTIMATED Estimates, to be replaced by the measurements of SUPER_SCALE step S8.

17.2 The deployment

Part	Size	Count
Server tasks	4 vCPU and 12 GB for the server (8 GB heap) in a 6 vCPU, 16 GB task with its console; 16 vCPU, 64 GB at SUPER_SCALE volumes	3 to 6, sharded by table
Loader task	4 vCPU, 30 GB, 21 GB ephemeral	one per table at load time
Maintenance and report tasks	the same image and size	nightly, minutes
EFS	each node's state and caches: 50–300 GB; loader scratch: one day of one table	one file system
A node reading the risk lake	heap ≈ 1.8 GB per open VaR date at 2.3M; hot files ≈ 5.5 GB a date on local NVMe (EC2)	≥ 1; whole dates each

17.3 Monthly cost (list prices, one Region, estimates)

Item	Basis	About
S3 storage, 200 TB tiered	10 TB Standard (USD 23/TB), 46 TB Standard-IA (12.5), 144 TB Glacier IR (4)	USD 1,400
S3 storage if all were Standard	200 TB × 23	USD 4,600
S3 requests	250 reads a second, two GETs each, USD 0.40 per million; loads and lists	USD 500–700
Retrievals from IA and Glacier IR	month-ends, audits, history	tens to a few hundred
Versioning overhead	14 days of replaced and vacuumed files	a few % of storage
Server nodes	3 tasks of 6 vCPU, 16 GB on Fargate, always on	USD 650–750
Loader, maintenance	about an hour a day	USD 50–150
EFS	1 TB Standard, Elastic throughput	USD 300 + throughput
CloudWatch, Secrets Manager, Cloud Map, endpoints, load balancer		USD 200–400

For comparison: the same 200 TB on EFS: USD 60,000 a month in Standard; about USD 3,200 in IA plus USD 0.03 per GB read.

18
Runbooks

The daily load, a backfill, a restatement, a slow table, a restated risk date, and disaster recovery

18.1 The daily load

Check	How
It ran	CloudWatch Logs `/drishti/<env>`, stream prefix loader; `LoadSucceeded` 1 per table
It published	`report.sh --tables <table>`: newest is today; in the console, `TRD <id>` on today's date; `DATAMAP` counts the day
Exit 2 (rejects)	the build summary names the reasons; fix the source and restate the day (18.2)
Exit 1	`lake build failed` or `lake publish failed` says why; the S3 table is unchanged; fix, then `load-day.sh --date <d> --tables <t>`
Exit 3	maintenance or a backfill held the table's lock; the next run loads it, or run it by hand after

18.2 A backfill and a restatement

A backfill

1. `backfill.sh --dry-run --from … --to … --tables <t>`: check the dates and commands
2. Run it as a loader task per table (`aws ecs run-task … --overrides` with `backfill.sh …`)
3. It stops at a failing date with the resume command; fix and resume
4. `setup-lifecycle.sh` (or the next maintain.sh) moves the backfilled years to their tiers

A restatement

1. The source re-sends the JSON Lines dumps under the landing prefix of the day
2. `backfill.sh --dates <d>` (whole day) or `load-day.sh --date <d> --tables <t> --mode overwrite-partitions`
3. Readers switch at the commit; the data-load signal (if set) refreshes the servers at once

18.3 A slow table

Symptom	Cause	Fix
first read of a day slow, then fast	id map not yet on disk; footers cold	normal; `warm-dates` (3) warms the newest days at start
every read of a day slow	the day is not sorted by id, or row groups are large	restate the day after fixing the order (7.2)
searches slow or partial	a field the search uses is not promoted	promote it; publish then refuses the next day: backfill into `trade_endur_v2` and switch `table.trade`
the snapshot refresh slow (Health)	many live files or a long log (intraday commits)	report.sh: behind and file counts; run maintain.sh; split the table
503 Slow Down	one prefix overloaded	partition by `PRODUCT_TYPE` (or finer); S3 scales a prefix within minutes
searches read documents on one table	deletion vectors in its files (written by Spark)	compaction rewrites the files without them

18.4 A restated risk date

1. The risk batch rewrites the whole date (a new `BATCH_ID`, `RESTATEMENT` + 1) as JSON Lines
2. Load it like any table: `load-day.sh --date <d> --tables risk/pnl-vector` (overwrite-dates)
3. The data-load signal reloads the date: a new hot file (new inputs' digest); the intraday layer is dropped
4. The official capture runs again: the next `RESULT_VERSION`, appended; the earlier version is kept
5. Check: `drishti.py var official` lists both versions; BI shows the latest; `publishError` empty

If the date did not load

`DRS-5203` (two runs or scenario sets on one date): the engine answers the date before, with a warning. Fix the batch, rewrite the date whole.

Nothing is lost

Time travel keeps the old inputs 7 years; the official table keeps every result version; a capture keeps its vector. "What did we report?" and "what is the corrected number?" both stay answerable.

18.5 Disaster recovery

Event	How to recover
A table damaged (a wrong commit)	Delta time travel: the previous version is readable until vacuum (7 years for risk); restate the days from the landing bucket (30 days) or from versions
Files deleted by mistake	S3 versioning: restore the noncurrent versions within `NONCURRENT_KEEP_DAYS` (list-object-versions, copy-object)
A Region lost	S3 Cross-Region Replication with RTC, EFS and ECR replication. A commit can arrive before a data file it names: that day fails (Health DEGRADED) until the file arrives. Fail over once the backlog is empty
Node state lost	AWS Backup of EFS; caches and hot files rebuild from the lake by themselves
Cluster state (content store)	s3 backend: the bucket's versioning and replication; filesystem on EFS: AWS Backup and EFS replication

19
The scripts

Every file under deploy/, the lake tools beside them, and the conventions every script follows

19. deploy/ and tools/lake/: the files


```
deploy/
├── docker/              compose, server and console Dockerfiles
│   └── ecs/             server, console, loader images + entry points
├── aws/
│   ├── env.example      every setting; copy to deploy/aws/env
│   ├── lib/common.sh    --help, --dry-run, --env, templates
│   ├── delta-lake/      setup-buckets, setup-iam,
│   │                    check-conditional-writes, setup-lifecycle,
│   │                    setup-alarms, load-day, backfill, maintain,
│   │                    report, lake-maintenance.s3.yaml, iam/
│   └── ecs/             setup-efs, setup-cloud-map, register,
│                        templates/*.json
├── grafana/             the Grafana dashboard
└── lake-maintenance.yaml   lake maintain's local example
```

```
tools/lake/
├── build.py  publish.py  maintain.py
├── demo.py   stage.py  writers.py
├── stores.py  stores_equality.py
├── s3sim.py                (2.7.0)
└── test_publish  test_maintain
    test_s3_simulators  test_vectors
    test_retention  test_names
tools/samplegen/load_rows.py (2.9.4)
tools/convertcmd.py   drishti.py convert
```

`drishti.py lake publish` (TOOLS.md §4.11) and the S3 tests (§4.12) run in the drill; `tools/test_deploy_aws.py` runs every script with --help and --dry-run.

19. Conventions every script follows

- bash with `set -euo pipefail` and the licence header
- AWS CLI v2 and nothing else; Python only through `tools/drishti.py`
- `--help` on every script; `--dry-run`: each changing call printed as `+ aws …`, each read as `? aws …`, none made
- Settings from one file: `deploy/aws/env` (from env.example; placeholders only; git-ignored), or `--env <file>`
- Safe to run again (idempotent): buckets, policies, access points and rules are kept and their settings applied again
- A placeholder left stops the script and names it
- Locks: load, backfill and maintenance take the table's flock on EFS; a held lock exits 3
- Data in is JSON Lines (2.9.4); a CSV source is converted first with `drishti.py convert`
- Tested: `tools/test_deploy_aws.py` (every script with --help and --dry-run against a fake settings file and an aws that fails if called; every template rendered; lifecycle rules; env git-ignored); `test_publish.py` and the S3 simulator tests run in the drill

20

Limits, decisions and what comes next

What is not yet proven, known limits with their sources, the decisions to take, and the documents behind every slide

20. Limits and what comes next

Limit	Status	Source
Not yet run in a real AWS account (steps 0–11, runbooks)	to do	AWS_DELTA_LAKE.md §15
Official results and Kafka windows are written by the Java lake writer: local or EFS root, not `s3://`	limit; to S3 PROPOSED	VAR_GUIDE.md §19, §8.4
VaR and exposure inputs read from Delta only (streamed vectors)	limit; Iceberg PROPOSED	SourcePlugin.vectors
The 1 M × 50 VaR benchmark (and 2.3M) on a real server	to do	VAR_GUIDE.md §27.3
More than 100,000 hierarchy nodes or portfolios from a connector that cannot list a date whole (15,000 portfolios fit)	limit	VAR_GUIDE.md §20
VaR sharding; the bank-scale vector layout	DESIGNED	VAR_GUIDE.md §28, §29.2
Tables by year, range maps, slice ownership (S1–S8)	DESIGNED	SUPER_SCALE.md
`lake build` sorts a partition only when it fits a batch	limit	AWS_DELTA_LAKE.md §7.2
A bucket without conditional writes: one writer per table	limit	AWS_DELTA_LAKE.md §8.2

Decisions and actions for the product owner

#	Decision or action	Recommendation
1	Disk caches and VaR/exposure hot files on local NVMe (breaks the identical-path rule) or EFS	Hot files on NVMe from the start (the guides ask for fast local disk); connector caches on EFS until p99Ms shows EFS latency
2	First real run in a development AWS account (steps 0–11, runbooks 18.1–18.5)	Before any production commitment; record the outputs in the guide
3	How official results and Kafka windows reach S3	EFS root, then `lake publish --mode append` with 7-year retention (PROPOSED)
4	A fixed gossip port on ECS	A number in node.yaml and a security-group rule node to node (PROPOSED)
5	Compaction on S3	Re-enable after a relayout test on one S3 table; update AWS_DELTA_LAKE.md 9.3 (PROPOSED)
6	Tables by year (S2); range maps, Bloom filters (S1); slice ownership (S5, S6); VaR sharding	When live files near 100,000 per table, or one server's working set or one VaR date becomes the limit

Related documents

Document	What it holds
docs/architecture/AWS_DELTA_LAKE.md	the first source: every section, command and number of the AWS lake
docs/architecture/AWS_DEPLOYMENT.md · SUPER_SCALE.md · SCALE_STRATEGY.md	ECS, EFS and the identical-path rule; 500 GB a day for 7 years (S1–S8); one server at millions a day
docs/connectors/DELTA_CONNECTOR.md	how Drishti reads a Delta table; layout, vectors, history, memory, measured
docs/admin/LAKE_MAINTENANCE.md · docs/guides/TOOLS.md	lake maintain and the 7-year floor; lake build, publish, the S3 tests
docs/admin/NET.md · docs/architecture/CONTENT_STORE.md	Drishti*net: gossip port, HTTP/2, routing; the content store on S3
docs/guides/DATA_FORMATS.md	JSON Lines is the standard; CSV deprecated; official results as JSONL
docs/guides/VAR_GUIDE.md · EXPOSURE_GUIDE.md	the risk lake: inputs, vectors, intraday, official results, sizes
docs/connectors/DEMO_DATA.md · docs/guides/PUBLIC_DATA.md	the 84-table demo lake; the public-data packs
Drishti-AWS-Iceberg.pptx · Drishti-VaR-Engine.pptx	the companion decks: Iceberg on AWS; the VaR engine